

Name: Roosc Zaño

Day 5

Student Enrollment Database – Design, Build & Query

Part 1 – Schema Design & Creation

Name	Value
Updated Rows	0
Execute time	0.006s
Start time	Mon May 11 14:55:13 PST 2026
Finish time	Mon May 11 14:55:13 PST 2026
Query	CREATE TABLE students (id SERIAL PRIMARY KEY, first_name VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL, year INT CHECK (year >= 1 AND year <= 4), gender VARCHAR(50), enrolled BOOLEAN DEFAULT TRUE, created_at TIMESTAMPTZ DEFAULT NOW())

Create students table

Create programs table

Name	Value
Updated Rows	0
Execute time	0.005s
Start time	Mon May 11 14:52:43 PST 2026
Finish time	Mon May 11 14:52:43 PST 2026
Query	CREATE TABLE programs (id SERIAL PRIMARY KEY, program_name VARCHAR(150) UNIQUE NOT NULL)

Create sections table

Statistics 1	×
Name	Value
Updated Rows	0
Execute time	0.005s
Start time	Mon May 11 14:59:00 PST 2026
Finish time	Mon May 11 14:59:00 PST 2026
Query	create table sections(id serial primary key, code varchar(100) not null unique, year int not null, program_id int not null references programs(id))

Part 2 – Seed Data & INSERT

Create student_sections table

Statistics 1	
Name	Value
Updated Rows	0
Execute time	0.005s
Start time	Mon May 11 14:59:54 PST 2026
Finish time	Mon May 11 14:59:54 PST 2026
Query	create table student_sections (id serial primary key, student_id int not null references students(id), section_id int not null references sections(id), enrolled_at timestamptz default now())

Create student_sections table

Statistics 1	
Name	Value
Updated Rows	0
Execute time	0.007s
Start time	Mon May 11 15:00:25 PST 2026
Finish time	Mon May 11 15:00:25 PST 2026
Query	create table student_grades(id serial primary key, student_id int not null references students(id), grade int check (grade >=0 and grade <= 100))

Insert into students with returning id

The screenshot shows a SQL editor with the following query:

```
insert into students (first_name, last_name, year, gender, enrolled)
values
('Roosc', 'Zaño', 3, 'Male', true),
('Ron Vincent', 'Cada', 3, 'Female', true),
('Mike Andrei', 'Gomez', 4, 'Female', false),
('Kurt Patrick', 'Laja', 2, 'Male', true)
returning id
```

Below the editor, the results pane is titled "students 1". It shows a table with the following data:

	123 id
1	5
2	6
3	7
4	8

Insert into programs with returning id

```
insert into programs (program_name)
values
('BS in Information Technology'),
('BS in Computer Science'),
('BS in Entertainment and Multimedia Computing')
returning id
```

programs 1 X

insert into programs (program_name) values ('BS in Inform | Enter a SQL expression to filter results (

	123 id
1	1
2	2
3	3

Insert into sections with returning id

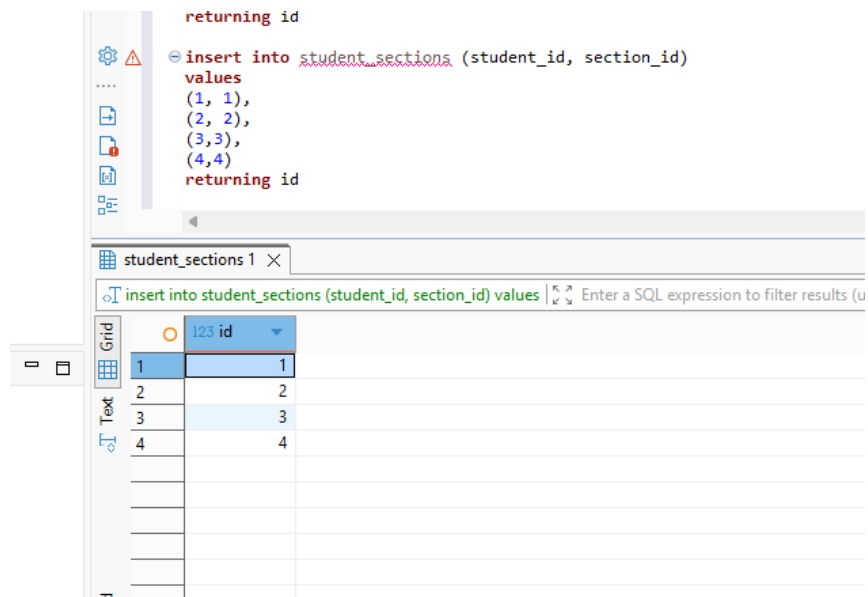
```
insert into sections (code, year, program_id)
values
('M001', 2020, 1),
('M002', 2022, 2),
('M003', 2022, 3),
('M004', 2021, 1)
returning id
```

sections 1 X

insert into sections (code, year, program_id) values ('M001 | Enter a SQL expression to filter results (use Ctrl+Space)

	123 id
1	1
2	2
3	3
4	4

Insert into student_sections with returning id



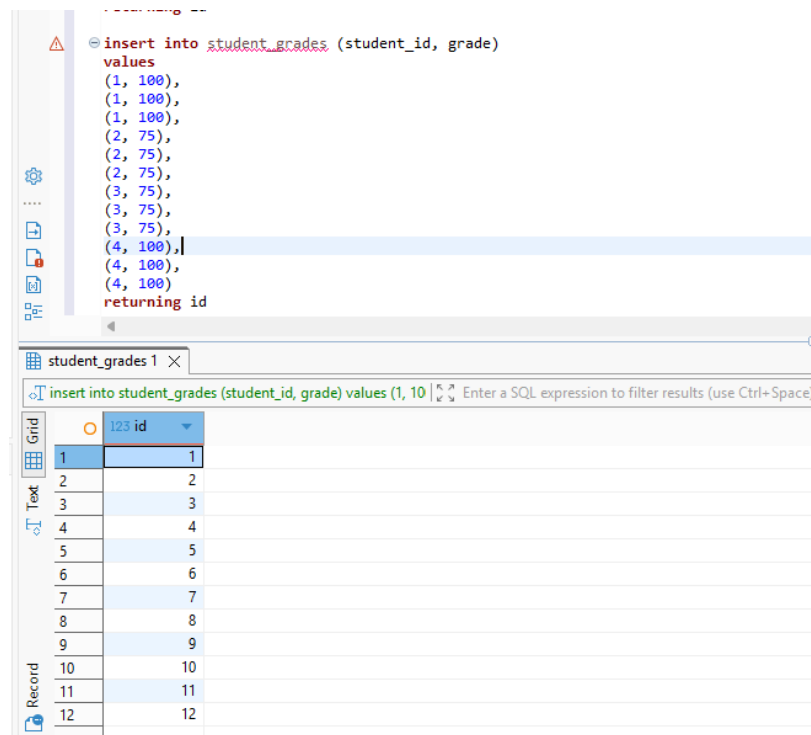
The screenshot shows a database client interface. At the top, an SQL statement is entered in a text area:

```
returning id  
insert into student_sections (student_id, section_id)  
values  
(1, 1),  
(2, 2),  
(3,3),  
(4,4)  
returning id
```

Below the text area, a tab labeled "student_sections 1" is active. The results of the query are displayed in a grid view. The grid has two columns: "id" and "section_id". The first four rows contain the data returned by the query.

id	section_id
1	1
2	2
3	3
4	4

Insert into student_grades with returning id



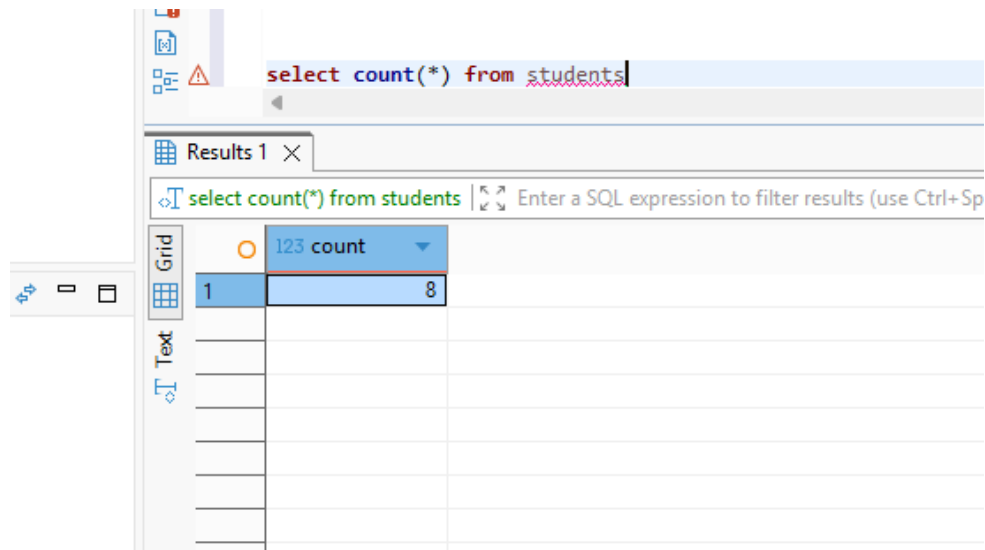
The screenshot shows a database client interface. At the top, an SQL statement is entered in a text area:

```
insert into student_grades (student_id, grade)  
values  
(1, 100),  
(1, 100),  
(1, 100),  
(2, 75),  
(2, 75),  
(2, 75),  
(3, 75),  
(3, 75),  
(3, 75),  
(4, 100),  
(4, 100),  
(4, 100)  
returning id
```

Below the text area, a tab labeled "student_grades 1" is active. The results of the query are displayed in a grid view. The grid has two columns: "id" and "grade". The first 12 rows contain the data returned by the query.

id	grade
1	100
2	100
3	100
4	75
5	75
6	75
7	75
8	75
9	75
10	100
11	100
12	100

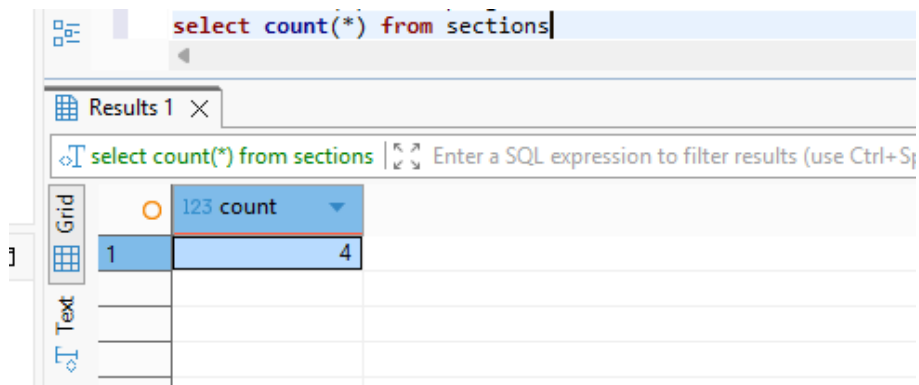
Select Count(*) from students



The screenshot shows a SQL query editor with the query `select count(*) from students` entered. Below the query, the results are displayed in a grid view. The grid has two columns: an index column and a column labeled 'count'. The first row shows the value 8.

	count
1	8

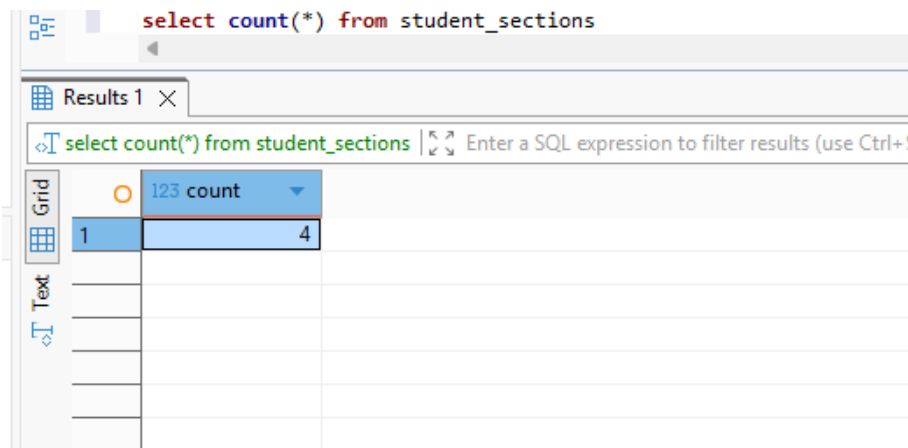
Select Count(*) from sections



The screenshot shows a SQL query editor with the query `select count(*) from sections` entered. Below the query, the results are displayed in a grid view. The grid has two columns: an index column and a column labeled 'count'. The first row shows the value 4.

	count
1	4

Select Count(*) from student_sections



The screenshot shows a SQL query editor with the query `select count(*) from student_sections` entered. Below the query, the results are displayed in a grid view. The grid has two columns: an index column and a column labeled 'count'. The first row shows the value 4.

	count
1	4

Select Count(*) from student_grades

The screenshot shows the SQL Developer interface. The query editor at the top contains the SQL statement: `select count(*) from student_grades`. Below the editor, the 'Results 1' window is open, displaying the query results in a grid format. The grid has two columns: 'count' and a value. The first row shows a count of 12.

	count
1	12

Part 3 – Queries

Display students by their assigned program names

The screenshot shows the SQL Developer interface with a complex query in the editor. The query is: `select s.id, s.first_name, s.last_name, s.year, s.gender, p.program_name from students s join student_sections ss on s.id = ss.student_id join sections sec on ss.section_id = sec.id join programs p on sec.program_id = p.id`. The 'students(+)' window displays the results in a grid format. The grid has seven columns: 'id', 'first_name', 'last_name', 'year', 'gender', and 'program_name'. There are four rows of data.

	id	first_name	last_name	year	gender	program_name
1	1	Roosc	Zaño	3	Male	BS in Information Technology
2	2	Ron Vincent	Cada	3	Female	BS in Computer Science
3	3	Mike Andrei	Gomez	4	Female	BS in Entertainment and Multimedia Computing
4	4	Kurt Patrick	Laja	2	Male	BS in Information Technology

Display the number of students per section using GROUP BY

The screenshot shows a database query tool interface. At the top, a SQL query is entered in a text area:

```
select sec.code as section_code, count(ss.student_id) as total_students
from sections sec
left join student_sections ss on sec.id = ss.section_id
group by sec.code
order by sec.code
```

Below the query, the results are displayed in a grid view. The grid has two columns: 'section_code' and 'total_students'. The results are as follows:

	section_code	total_students
1	M001	1
2	M002	1
3	M003	1
4	M004	1

Count the number of students per program

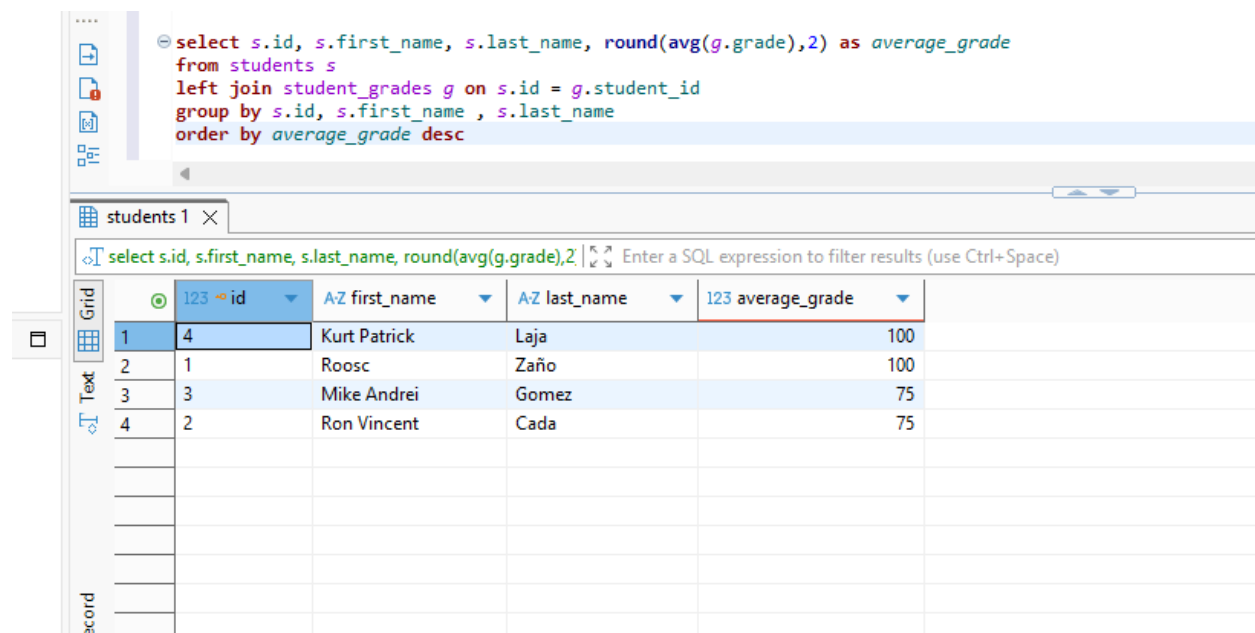
The screenshot shows a database query tool interface. At the top, a SQL query is entered in a text area:

```
select p.program_name, count(ss.student_id) as total_students
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
group by p.program_name
order by p.program_name
```

Below the query, the results are displayed in a grid view. The grid has two columns: 'program_name' and 'total_students'. The results are as follows:

	program_name	total_students
1	BS in Computer Science	1
2	BS in Entertainment and Multimedia Computing	1
3	BS in Information Technology	2

Display students by their grades in descending order



The screenshot shows a SQL IDE with a query editor at the top and a results grid below. The query is:

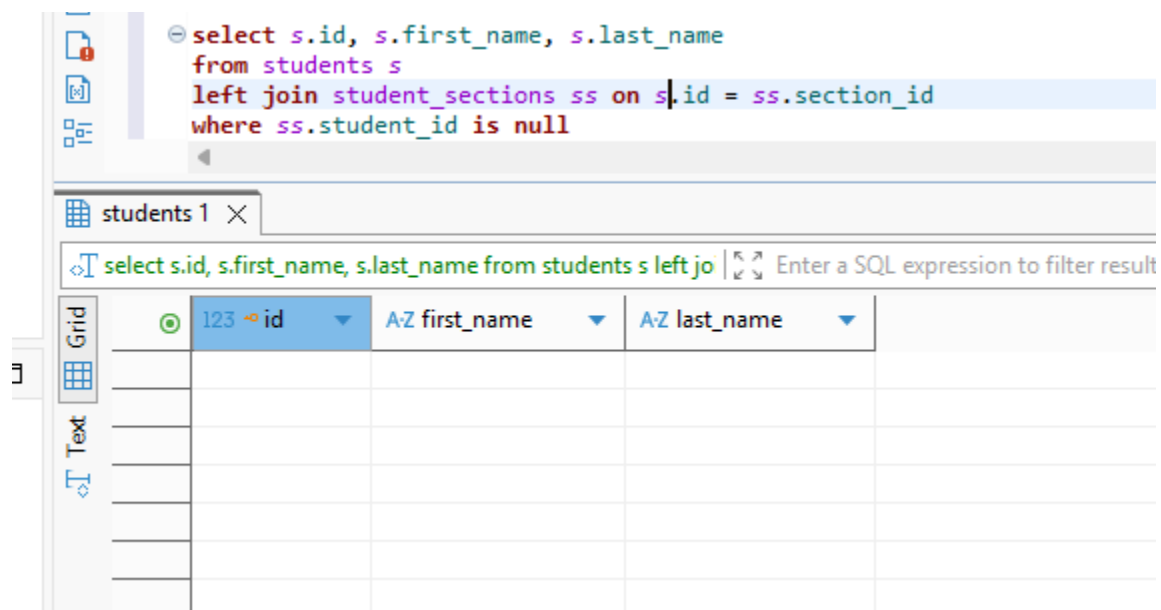
```
select s.id, s.first_name, s.last_name, round(avg(g.grade),2) as average_grade
from students s
left join student_grades g on s.id = g.student_id
group by s.id, s.first_name, s.last_name
order by average_grade desc
```

The results grid, titled "students 1", displays the following data:

	id	first_name	last_name	average_grade
1	4	Kurt Patrick	Laja	100
2	1	Roosc	Zaño	100
3	3	Mike Andrei	Gomez	75
4	2	Ron Vincent	Cada	75

Finding students whose section_id is null

In this example, there is no student without section_id



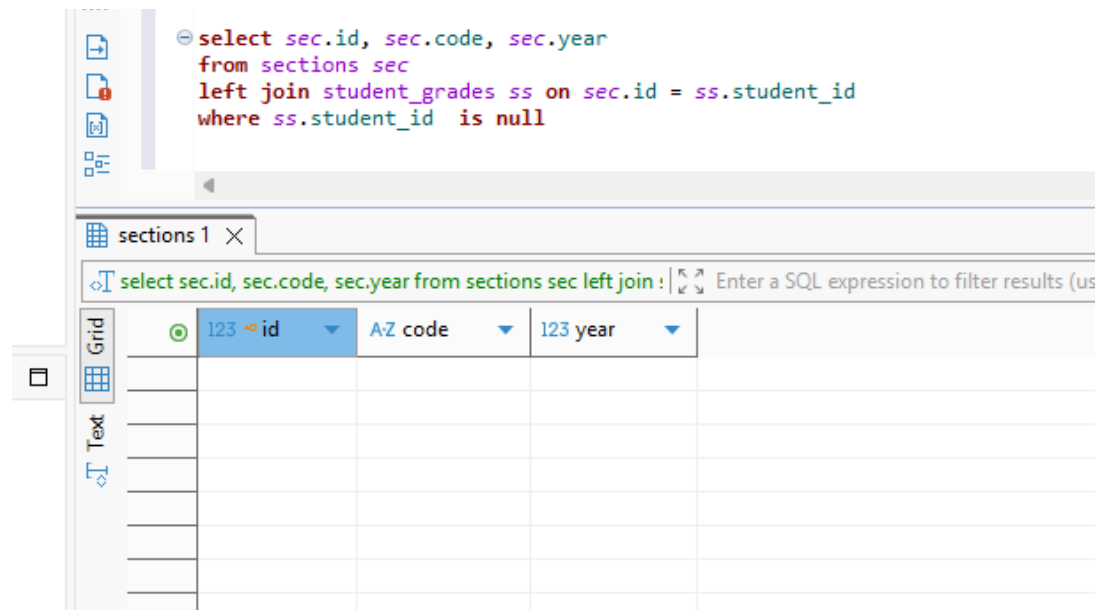
The screenshot shows a SQL IDE with a query editor at the top and a results grid below. The query is:

```
select s.id, s.first_name, s.last_name
from students s
left join student_sections ss on s.id = ss.section_id
where ss.student_id is null
```

The results grid, titled "students 1", is currently empty, indicating that no students were found without a section_id.

Finding students whose section_id is null

In this example, there is no student without a student_id



The screenshot shows a SQL IDE interface. At the top, a query editor contains the following SQL code:

```
select sec.id, sec.code, sec.year
from sections sec
left join student_grades ss on sec.id = ss.student_id
where ss.student_id is null
```

Below the query editor, a tab labeled "sections 1" is active. The results pane shows a table with the following columns: "id", "code", and "year". The first row of data is highlighted, showing the values "123", "A-Z", and "123".

id	code	year
123	A-Z	123

Part 4 – Indexes & Constraints

```
explain analyze
select s.first_name, s.last_name, p.program_name, sec.code as section_code, round(avg(sg.grade), 2) as average_grade
from students s
join student_sections ss on s.id = ss.student_id
join sections sec on ss.section_id = sec.id
join programs p on sec.program_id = p.id
left join student_grades sg on s.id = sg.student_id
group by s.id, s.first_name, s.last_name, p.program_name, sec.code
order by s.id
```

before creating index

explain analyze select s.first_name, s.last_name, p.program Enter a SQL expression to filter results (use Ctrl+Space)	
Grid	AZ QUERY PLAN
9	Hash Cond: (s.id = sg.student_id)
10	Buffers: shared hit=5
11	-> Nested Loop (cost=2.18..27.46 rows=4 width=976) (actual time=0.095..0.102 rows=4.00 loops=1)
12	Join Filter: (sec.id = ss.section_id)
13	Rows Removed by Join Filter: 12
14	Buffers: shared hit=4
15	-> Hash Join (cost=1.09..12.92 rows=4 width=444) (actual time=0.057..0.059 rows=4.00 loops=1)
16	Hash Cond: (s.id = ss.student_id)
17	Buffers: shared hit=2
18	-> Seq Scan on students s (cost=0.00..11.30 rows=130 width=440) (actual time=0.029..0.029 rows=130.00 loops=1)
19	Buffers: shared hit=1
20	-> Hash (cost=1.04..1.04 rows=4 width=8) (actual time=0.017..0.017 rows=4.00 loops=1)
21	Buckets: 1024 Batches: 1 Memory Usage: 9kB
22	Buffers: shared hit=1
23	-> Seq Scan on student_sections ss (cost=0.00..1.04 rows=4 width=8) (actual time=0.012..0.012 rows=4.00 loops=1)
24	Buffers: shared hit=1
25	-> Materialize (cost=1.09..14.31 rows=4 width=540) (actual time=0.009..0.010 rows=4.00 loops=1)
26	Storage: Memory Maximum Storage: 17kB
27	Buffers: shared hit=2
28	-> Hash Join (cost=1.09..14.29 rows=4 width=540) (actual time=0.030..0.032 rows=4.00 loops=1)
29	Hash Cond: (p.id = sec.program_id)
30	Buffers: shared hit=2
31	-> Seq Scan on programs p (cost=0.00..12.30 rows=230 width=322) (actual time=0.007..0.007 rows=230.00 loops=1)
32	Buffers: shared hit=1
33	-> Hash (cost=1.04..1.04 rows=4 width=226) (actual time=0.014..0.014 rows=4.00 loops=1)
34	Buckets: 1024 Batches: 1 Memory Usage: 9kB
35	Buffers: shared hit=1
36	-> Seq Scan on sections sec (cost=0.00..1.04 rows=4 width=226) (actual time=0.008..0.008 rows=4.00 loops=1)
37	Buffers: shared hit=1
38	-> Hash (cost=1.12..1.12 rows=12 width=8) (actual time=0.013..0.013 rows=12.00 loops=1)
39	Buckets: 1024 Batches: 1 Memory Usage: 9kB
40	Buffers: shared hit=1
41	-> Seq Scan on student_grades sg (cost=0.00..1.12 rows=12 width=8) (actual time=0.008..0.009 rows=12.00 loops=1)
42	Buffers: shared hit=1
43	Planning Time: 0.439 ms
44	Execution Time: 0.273 ms
Record	

Create index

Name	Value
Updated Rows	0
Execute time	0.008s
Start time	Mon May 11 16:17:29 PST 2026
Finish time	Mon May 11 16:17:29 PST 2026
Query	create index idx_sections_program_id on sections(program_id); create index idx_student_sections_student_id on student_sections(student_id); create index idx_student_sections_section_id on student_sections(section_id); create index idx_student_grades_student_id on student_grades(student_id)

After creating index

		Grid Text Record	AZ QUERY PLAN
6	Sort Method: quicksort Memory: 25kB		
7	Buffers: shared hit=11 read=1		
8	-> Nested Loop Left Join (cost=2.32..28.48 rows=4 width=980) (actual time=0.205..0.221 rows=12.00 loops=1)		
9	Buffers: shared hit=11 read=1		
10	-> Nested Loop (cost=2.18..27.46 rows=4 width=976) (actual time=0.131..0.140 rows=4.00 loops=1)		
11	Join Filter: (ss.section_id = sec.id)		
12	Rows Removed by Join Filter: 12		
13	Buffers: shared hit=4		
14	-> Hash Join (cost=1.09..12.92 rows=4 width=444) (actual time=0.043..0.046 rows=4.00 loops=1)		
15	Hash Cond: (s.id = ss.student_id)		
16	Buffers: shared hit=2		
17	-> Seq Scan on students s (cost=0.00..11.30 rows=130 width=440) (actual time=0.017..0.018 rows=4.00 loops=1)		
18	Buffers: shared hit=1		
19	-> Hash (cost=1.04..1.04 rows=4 width=8) (actual time=0.013..0.014 rows=4.00 loops=1)		
20	Buckets: 1024 Batches: 1 Memory Usage: 9kB		
21	Buffers: shared hit=1		
22	-> Seq Scan on student_sections ss (cost=0.00..1.04 rows=4 width=8) (actual time=0.010..0.011 rows=4.00 loops=1)		
23	Buffers: shared hit=1		
24	-> Materialize (cost=1.09..14.31 rows=4 width=540) (actual time=0.021..0.023 rows=4.00 loops=4)		
25	Storage: Memory Maximum Storage: 17kB		
26	Buffers: shared hit=2		
27	-> Hash Join (cost=1.09..14.29 rows=4 width=540) (actual time=0.074..0.077 rows=4.00 loops=1)		
28	Hash Cond: (p.id = sec.program_id)		
29	Buffers: shared hit=2		
30	-> Seq Scan on programs p (cost=0.00..12.30 rows=230 width=322) (actual time=0.019..0.020 rows=4.00 loops=1)		
31	Buffers: shared hit=1		
32	-> Hash (cost=1.04..1.04 rows=4 width=226) (actual time=0.016..0.016 rows=4.00 loops=1)		
33	Buckets: 1024 Batches: 1 Memory Usage: 9kB		
34	Buffers: shared hit=1		
35	-> Seq Scan on sections sec (cost=0.00..1.04 rows=4 width=226) (actual time=0.012..0.012 rows=4.00 loops=1)		
36	Buffers: shared hit=1		
37	-> Index Scan using idx_student_grades_student_id on student_grades sg (cost=0.14..0.24 rows=1 width=8)		
38	Index Cond: (student_id = s.id)		
39	Index Searches: 4		
40	Buffers: shared hit=7 read=1		
41	Planning:		
42	Buffers: shared hit=66 read=4 dirtied=3		
43	Planning Time: 4.064 ms		
44	Execution Time: 0.422 ms		